EXERCÍCIO 01 / CONTRIBUINDO EM UM PROJETO

Issues, Forks, Branches e Pull Requests

Você fará uma contribuição individual para um repositório compartilhado. Seu arquivo será incorporado ao projeto após a revisão e o merge do Pull Request pelo instrutor.

REPOSITÓRIO DA ATIVIDADE
https://github.com/DaviCampos09/minicurso-git-colaboracao

O que você deve fazer

1. **Abra uma Issue no repositório original.**
Na aba **Issues**, crie a Issue **"Adicionar apresentação de @seu_usuario"** e descreva brevemente a alteração pretendida. Anote o número da Issue.

2. **Crie seu Fork e clone-o.**
Utilize **Fork** no repositório original para gerar uma cópia na **sua conta**. Em seguida, clone o **seu fork** no computador, não o repositório original.

3. **Crie uma branch para a contribuição.**
No repositório local, crie e acesse a branch `feature/seu_usuario`. Confirme que está nela antes de modificar arquivos.

4. **Adicione seu arquivo de apresentação.**
Dentro de `participantes/`, crie `seu_usuario.txt`. Escreva seu nome ou apelido, seu curso e uma breve mensagem sobre o minicurso. Não altere os arquivos dos colegas.

5. **Registre e envie a alteração.**
Utilize `git add` e `git commit` para registrar a contribuição. Depois, use `git push` para enviar sua **branch** ao fork no GitHub.

6. **Abra um Pull Request para o projeto original.**
Compare sua branch do fork com a `main` do repositório de **DaviCampos09**. Descreva a contribuição e inclua `Closes #N`, substituindo **N** pelo número da sua Issue.

7. **Aguarde a integração.**
Confira se o PR mostra somente a sua contribuição. **Não faça o merge:** o instrutor revisará os PRs, integrará os arquivos na `main` e mostrará o resultado à turma.

Critério de conclusão: sua Issue foi criada no projeto original e seu Pull Request foi aberto para a `main` desse mesmo projeto. Após o merge, o GitHub deverá fechar a Issue vinculada automaticamente.

EXERCÍCIO 02 / RESOLVENDO UM CONFLITO

Conflitos de Merge e Git Stash

Você trabalhará individualmente com duas branches que fazem alterações incompatíveis no mesmo arquivo. Toda a resolução será realizada no seu computador.

REPOSITÓRIO DA ATIVIDADE
https://github.com/DaviCampos09/minicurso-git-conflitos

O que você deve fazer

1. **Clone o repositório e confira a branch principal.**
Faça o clone do repositório indicado acima. O arquivo `conflito.txt` começa com a mensagem `Mensagem: Eu estou aprendendo Git.`

2. **Trabalhe na primeira branch.**
A partir da `main`, crie e acesse `teste/a`. No arquivo `conflito.txt`, troque a mensagem para `Mensagem: Git é um sistema distribuído.` Depois, faça `git add` e `git commit`.

3. **Trabalhe na segunda branch.**
Volte à `main` e, a partir dela, crie e acesse `teste/b`. Altere a **mesma linha** para `Mensagem: Git facilita a colaboração.` Depois, faça outro `git add` e `git commit`.

4. **Provoque o conflito.**
Permanecendo em `teste/b`, execute `git merge teste/a`. Observe o aviso do Git e abra `conflito.txt` para identificar os trechos conflitantes.

5. **Resolva manualmente.**
Edite o arquivo para produzir uma **única mensagem final coerente**, removendo os marcadores de conflito (<<<<<<<, ======= e >>>>>>>).

6. **Finalize e verifique o merge.**
Adicione o arquivo corrigido à Staging Area e finalize o merge com um commit. Confira `git status` e consulte o histórico de commits para verificar o resultado.

Atenção: não execute `git push` nesta atividade. As branches `teste/a` e `teste/b` serão locais; por isso, todos podem usar esses mesmos nomes sem interferir nos colegas.

Desafio extra (opcional) — Git Stash

Depois de resolver o conflito, modifique o arquivo `README.md` **sem realizar commit**. Use `git stash` para guardar temporariamente a alteração, consulte `git stash list` e confira o estado do repositório. Em seguida, use `git stash pop` e verifique se a alteração reapareceu.

Critério de conclusão: o conflito foi resolvido, o merge foi finalizado e o repositório está sem conflitos pendentes. O desafio extra apenas reforça o uso do Stash.